

Streams (સ્ટ્રીમ્સ)

- stream એ ડેટાની sequence છે. જાવામાં stream એ bytesથી બનેલી હોય છે.
- જાવામાં, આપમેળે 3 સ્ટ્રીમ્સ બનાવવામાં આવે છે.
 1. **System.out** : standard output stream
 2. **System.in** : standard input stream
 3. **System.err** : standard error stream

Byte Streams

- જાવા બાઇટ સ્ટ્રીમ્સનો ઉપયોગ બાઈનરી ફાઇલમાંથી એક સમયે 8-બિટ્સ / 1 બાઇટના ઇનપુટ અને આઉટપુટને કરવા માટે થાય છે.
- InputStream અને OutputStream class એ બાઇટ સ્ટ્રીમનો સૌથી વધુ ઉપયોગમાં લેવાતા class છે.

Byte Streams classes

Stream class	Description
BufferedInputStream	Buffered input stream માટે use થાય છે.
BufferedOutputStream	Buffered output stream માટે use થાય છે.
DataInputStream	જાવા સ્ટાન્ડર્ડ ડેટા type reading કરવા માટેની method હોય છે.
DataOutputStream	જાવા સ્ટાન્ડર્ડ ડેટા type writing કરવા માટેની method હોય છે.
FileInputStream	ઇનપુટ સ્ટ્રીમ જે ફાઇલમાંથી read કરે છે.
FileOutputStream	આઉટપુટ સ્ટ્રીમ જે ફાઇલમાં લખે છે.
InputStream	એબ્સ્ટ્રેક્ટ class કે જે સ્ટ્રીમ ઇનપુટ ને describe કરે છે.
OutputStream	એબ્સ્ટ્રેક્ટ class કે જે સ્ટ્રીમ આઉટપુટ ને describe કરે છે.
PrintStream	આઉટપુટ સ્ટ્રીમ જેમાં print () અને println () method હોય છે.

FileOutputStream

- FileOutputStreamનો ઉપયોગ ફાઇલ બનાવવા અને તેમાં ડેટા લખવા માટે થાય છે.
- OutputStream એ file open કરતા પહેલા check કરે છે કે file already exist છે કે નહિ જો નથી તો તે file ને create કરે છે.
- FileOutputStream સાથે ફાઇલનો ઉપયોગ કરવાની બે રીત છે.

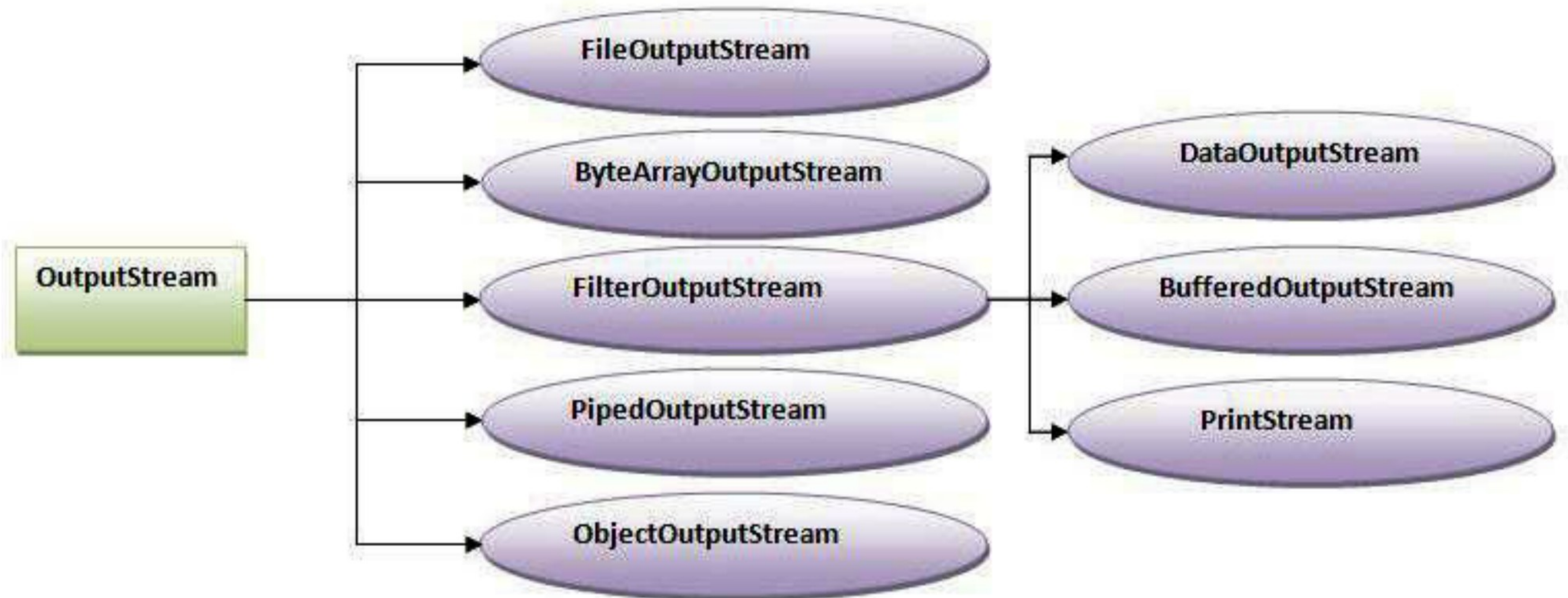
1. **FileOutputStream f = new FileOutputStream("C:/java/hello.txt");**

✓ અહીં, FileOutputStream નો object બનાવતી વખતે સીધા ફાઇલ પાથ specified કરવામાં આવે છે.

2. **File f = new File("C:/java/hello.txt");**

FileOutputStream f1 = new FileOutputStream(f);

✓ બીજી method, પ્રથમ File નો object બનાવવામાં આવે છે અને પછી ફાઇલ FileOutputStream નો object બનાવવામાં આવે છે અને File નો object ને FileOutputStream પર argument તરીકે pass કરવામાં આવે છે.



FileInputStream

- FileInputStream નો use ફાઇલોમાંથી ડેટા read કરવા માટે થાય છે.
- FileInputStream class નો ઉપયોગ કરતી વખતે, ફાઇલ exist હોવી આવશ્યક છે જો ફાઇલ exist ન હોય તો તે error generate કરશે.
- FileInputStream સાથે ફાઇલનો ઉપયોગ કરવાની બે રીત છે.

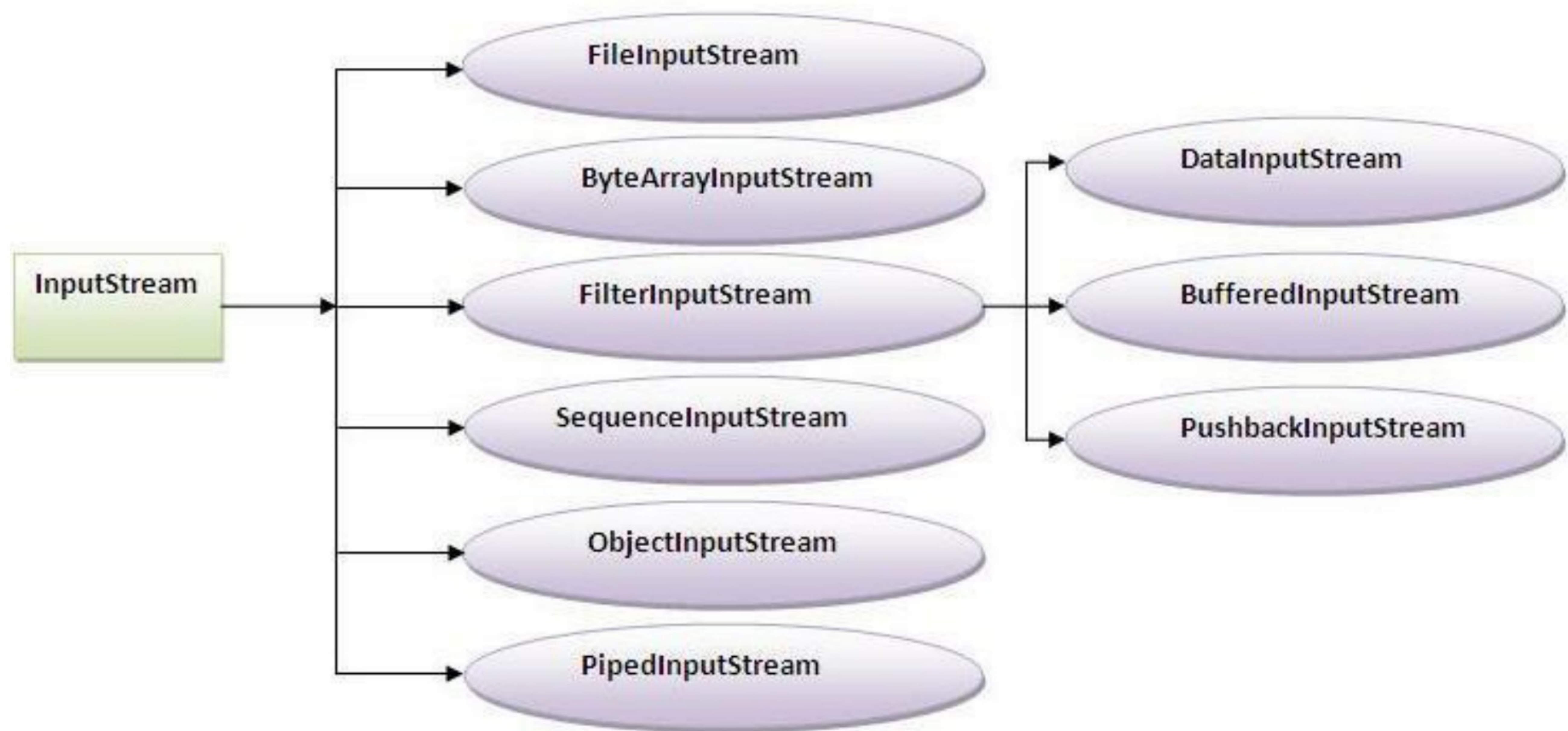
1. **FileInputStream f1 = new FileInputStream("C:/java/hello.txt");**

✓ અહીં, FileInputStream નો object બનાવતી વખતે સીધા ફાઇલ પાથ specified કરવામાં આવે છે.

2. **File f = new File("C:/java/hello.txt");**

FileInputStream f1 = new FileInputStream(f);

✓ બીજી method, પ્રથમ File નો object બનાવવામાં આવે છે અને પછી ફાઇલ FileInputStream નો object બનાવવામાં આવે છે અને File નો object ને FileInputStream પર argument તરીકે pass કરવામાં આવે છે.



Example : Write a java program that writes the Hello.txt file into Hello1.txt using FileInputStream and FileOutputStream.

Solution: File_1.java

```

import java.io.*;
public class File_1
{
    public static void main(String[] args)
    {
        try
        {
            FileInputStream in = new FileInputStream("C:/temp/Hello.txt ");
            FileOutputStream out = new FileOutputStream("C:/temp/Hello1.txt ");
            int c = 0;
            while ((c = in.read())!=-1)
            {
                out.write(c);
            }
            in.close();
            out.close();
            System.out.println("Successfully Write");
        }
        catch (FileNotFoundException e)
        {
            e.printStackTrace();
        }
        catch (IOException e)
  
```

```

    {
        e.printStackTrace();
    }
}

```

Output:

Successfully Write

Character Streams

- Character ફાઇલમાંથી એક સમયે 16-બીટ્સ / 2 બાઇટ્સ નું ઇનપુટ અને આઉટપુટ કરવા માટે Character streams નો ઉપયોગ થાય છે.
- Reader અને Writer એ character stream ના સૌથી વધુ ઉપયોગમાં લેવાતા class છે.

Character Stream classes

Stream class	Description
BufferedReader	Handle buffered input ને હેન્ડલ કરે છે.
BufferedWriter	Handle buffered output ને હેન્ડલ કરે છે.
FileReader	Input stream જે ફાઇલમાંથી read કરે છે.
FileWriter	Output stream જે ફાઇલમાંથી read કરે છે.
InputStreamReader	Input stream જે byte characterમાં translate કરે છે.
OutputStreamReader	Output stream જે byte characterમાં translate કરે છે.
Reader	Abstract class કે જે character stream input ને define કરે છે.
Writer	Abstract class કે જે character stream output ને define કરે છે.

FileWriter

- જાવા માં FileWriter ક્લાસનો ઉપયોગ ફાઇલમાં કેરેક્ટર ડેટા ને write કરવા માટે થાય છે.
- FileWriter ના Object ને બનાવવાની બે રીત.
 - FileWriter f = new FileWriter(String fname); //File path is specified in argument.
 - FileWriter f = new FileWriter(File f1); //1st file is created and it is passed as argument.

Method of FileWriter Class

void write(int ch)	ફાઇલ પર single character ને write કરે છે.
void write(String s)	FileWriter માં string ને લખે છે.

void write(char[] c)	FileWriter માં char array ને લખે છે.
void close()	FileWriter ને બંધ કરે છે.

FileReader

- જાવા માં FileReader ક્લાસનો ઉપયોગ ફાઇલમાં કેરેક્ટર ડેટા ને read કરવા માટે કરવામાં આવે છે.
- FileReader નો Object ને બનાવવાની બે રીત.
 - FileReader f = new FileReader(String name); //File path is specified in argument.
 - FileReader f = new FileReader(File f1); //1st file is created and it is passed as argument.

Method of FileReader Class

int read()	ASCII formમાં character Return કરે છે. તે ફાઇલના અંતે -1 Return કરે છે.
int read(char[] ch)	ફાઇલમાંથી ડેટાને char array માં read કરવા માટે.
void close()	FileReader ને બંધ કરે છે.

Example :Write a java program that writes the Hello.txt file into Hello1.txt using FileReader and FileWriter.

Solution: File_2.java

```
import java.io.*;
public class File_2
{
    public static void main(String[] args) throws IOException
    {
        File f = new File("d:/temp/Hello.txt");
        File f1 = new File("d:/temp/Hello1.txt");
        FileReader fr = new FileReader(f);
        FileWriter fw = new FileWriter(f1);
        int c = 0;
        while ((c = fr.read()) != -1)
        {
            fw.write(c);
        }
        fr.close();
        fw.close();
        System.out.println("Successfully Write");
    }
}
```

Output:

Successfully Write

BufferedReader

- BufferedReader class નો ઉપયોગ readLine () methodનો ઉપયોગ કરીને line by line ડેટા read કરવા માટે થાય છે.

BufferedOutputStream

- BufferedOutputStream class ડેટા સ્ટોર કરવા માટે internal buffer નો ઉપયોગ કરે છે.
- તે સીધા જ streamમાં ડેટા લખવા કરતાં વધુ efficiency add કરે છે. તેથી, તે performance ને fast બનાવે છે.

BufferedInputStream

- BufferedInputStream class સ્ટ્રીમમાંથી information ને read કરવા માટે વપરાય છે. તે performance ને fast બનાવવા માટે internally buffer mechanismનો ઉપયોગ કરે છે.

Example : Write a program in java that take character as a input from console and write it in file develop this program using Bufered Classes.

Solution: File_3.java

```
import java.io.*;
public class File_3
{
    public static void main(String[] args)
    {
        InputStreamReader in = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(in);
        File f = new File("d:/temp/Hello.txt");
        try
        {
            OutputStream out = new FileOutputStream(f);
            System.out.print("Type your text to write in file===");
            String str;
            str = br.readLine();
            byte b[] = str.getBytes();
            out.write(b);
            System.out.println("File Successfully Write..");
            br.close();
            in.close();
            out.close();
        }
    }
}
```

```
        catch (IOException e)
        {
            e.printStackTrace();
        }
    }
}
```

Output:

Type your text to write in file===i like an ice cream
File Successfully Write..